

AI & DATA SCIENCE STUDY SERIES

How Large Language Models (LLMs) Work

From Raw Text on the Internet to Intelligence in a Chatbot

Target Audience	Difficulty	Modules	Reading Time
Beginners — Intermediate	Beginner-Friendly	10 Core Modules	60–75 Minutes

What You Will Learn:

- Understand what an LLM is and where it fits in the AI landscape
- Learn how raw internet text becomes a trained language model — step by step
- Deeply understand tokenisation, embeddings, and the Transformer architecture
- Grasp the attention mechanism in plain English — the single most important concept in modern AI
- Follow the full training pipeline: pre-training, fine-tuning, RLHF, and alignment
- Understand inference: how the model generates text token by token
- Compare the world's leading LLMs across key dimensions
- Understand capabilities, limitations, hallucinations, and how to work around them
- Explore what comes next: scaling laws, reasoning models, and the road to AGI

No maths degree required. Every concept is explained with plain-English analogies and concrete examples. Technical depth is added progressively.

Module 1: What is a Large Language Model?

1.1 The Big Picture in One Sentence

A Large Language Model is a neural network trained on enormous amounts of text that learns to predict the next word — and in doing so, develops a surprisingly deep understanding of language, facts, reasoning, and the world.

KEY CONCEPT
LLM = A very large neural network, trained on a very large text dataset, to do one deceptively simple task: predict the next token.
Everything else — answering questions, writing code, summarising documents, reasoning through problems — emerges from this one training objective at sufficient scale.

1.2 Where LLMs Sit in the AI Landscape

Term	What It Means	Relationship
Artificial Intelligence (AI)	The broad field of making machines intelligent	LLMs are one type of AI
Machine Learning (ML)	AI systems that learn from data rather than explicit rules	LLMs are a type of ML
Deep Learning (DL)	ML using multi-layer neural networks	LLMs are a type of deep learning
Neural Network	A mathematical system loosely inspired by the brain	LLMs are a specific neural network architecture
Transformer	The specific neural network architecture used in LLMs	LLMs are Transformer-based models
Large Language Model (LLM)	A very large Transformer trained on text data	Our subject — the most specific level
Foundation Model	A large pre-trained model used as a base for many tasks	LLMs are the most prominent foundation models
Generative AI	AI that creates new content (text, images, code, etc.)	LLMs are the text branch of generative AI

1.3 Why 'Large'?

The word 'large' refers to the number of parameters — the numerical values the model learns during training. Parameters are the model's memory of everything it has learned.

Scale Milestone	Parameter Count & Significance
-----------------	--------------------------------

Early NLP models (2018)	BERT: 110 million parameters — impressive at the time, handles sentence understanding
GPT-2 (2019)	1.5 billion — OpenAI initially withheld it, fearing misuse
GPT-3 (2020)	175 billion — sparked the modern LLM era; in-context learning emerged
PaLM (2022)	540 billion — Google's model; showed continued scaling gains
GPT-4 (2023)	Estimated ~1 trillion (mixture of experts) — current commercial frontier
Llama 3.1 (2024)	405 billion — Meta's open-source frontier model
Key insight	More parameters ≠ always better; architecture, data quality, and training method matter equally

1.4 What LLMs Are Not

COMMON MISCONCEPTION

LLMs do NOT 'think' or 'understand' in the human sense. They are pattern-completion engines at massive scale.

LLMs do NOT access the internet in real time (unless given a search tool). They rely entirely on training data.

LLMs do NOT have memory between conversations by default — each conversation starts fresh.

LLMs do NOT always tell the truth. They generate plausible text, not necessarily accurate text.

LLMs are NOT just 'autocomplete'. The emergent capabilities at scale go far beyond simple next-word prediction.

1.5 The Most Important Analogy

ANALOGY

Imagine you read every book, article, forum post, and website ever written — billions of pages. You did not memorise them word-for-word. Instead, you absorbed patterns: how ideas connect, how sentences flow, how problems are solved, what facts appear together.

Now someone gives you the beginning of a sentence. You naturally know how to continue it — not because you looked it up, but because you have seen so many similar patterns.

That, at its core, is what an LLM does. The 'magic' is that when you do this at sufficient scale, the model develops capabilities that were never explicitly taught.

KEY TAKEAWAY

LLMs are next-token predictors trained at massive scale. Emergent intelligence is a side effect of scale applied to this simple objective.

Module 2: Tokenisation — How Text Becomes Numbers

Before a neural network can process text, the text must be converted into numbers. Tokenisation is this conversion step — and it shapes almost everything about how LLMs behave.

2.1 What is a Token?

KEY CONCEPT

A token is the basic unit of text that an LLM processes. Tokens are not always whole words — they are chunks of text determined by a tokeniser algorithm.

Rule of thumb: 1 token $\approx$ 0.75 English words, or roughly 4 characters.

100 tokens $\approx$ 75 words $\approx$ one short paragraph.

2.2 How Tokenisation Works

Modern LLMs use a method called Byte-Pair Encoding (BPE) or a variant. Here is how it works:

Step	What Happens
1. Start with characters	Begin with individual characters as the base vocabulary: a, b, c, ..., space, punctuation
2. Count pairs	Count every adjacent pair of characters in the training corpus: 'th', 'he', 'in', 'er', etc.
3. Merge most frequent pair	Merge the most common pair into a single token. 'th' becomes one token.
4. Repeat	Repeat thousands of times. Common words become single tokens; rare words split into sub-word pieces.
5. Final vocabulary	A vocabulary of 30,000–100,000 tokens covering common words whole and rare words as fragments

```
# Tokenisation Example (GPT-style)

Input:  'The transformer architecture revolutionised NLP.'

Tokens: ['The', ' transform', 'er', ' architecture',
         ' revolution', 'ised', ' NLP', '.']

Token IDs: [464, 11431, 263, 10959, 11331, 1417, 399, 43, 13]

# Note: 'transformer' splits into 'transform' + 'er'
# because 'transformer' is less common than its sub-parts
```

2.3 Why Tokenisation Matters for Practitioners

Implication	Practical Impact
Counting is in tokens, not words	Pricing, context limits, and speed are all measured in tokens — 1000 words ≈ 1333 tokens
Rare words split into sub-tokens	The model sees 'antidisestablishmentarianism' as 5+ tokens, increasing processing cost
Languages differ in token efficiency	English is highly efficient; some Asian languages use 2–4× more tokens for the same content
Numbers are often split oddly	'12345' may become ['123','45'] — this is why LLMs struggle with arithmetic on large numbers
Spelling sensitivity	The model sees letter sequences, not abstract symbols — typos genuinely confuse it at the token level
The space before a word matters	' dog' and 'dog' are different tokens — subtle prompt formatting can affect outputs

2.4 The Vocabulary and Embedding Step

Once text is tokenised, each token ID is converted into an embedding — a list of hundreds or thousands of numbers that represents the token's meaning and position in the vocabulary.

KEY CONCEPT

Each token gets a vector of typically 768 to 12,288 numbers (depending on model size). These vectors are not hand-crafted — they are learned during training. Similar tokens cluster together in vector space: 'king' is close to 'queen', 'man', 'monarch'. This is where language meaning first enters the model — before any attention is applied.

KEY TAKEAWAY

Text → Tokens (integer IDs) → Embeddings (dense float vectors). Only after this conversion does the Transformer begin processing.

Module 3: The Transformer Architecture

The Transformer, introduced in the landmark 2017 paper 'Attention Is All You Need', is the architectural foundation of every modern LLM. Understanding it is the single most valuable thing you can do to understand AI today.

3.1 The High-Level View

Component	Role in Plain English
Input Embeddings	Convert each token into a rich numerical vector representing its meaning
Positional Encoding	Add information about where each token sits in the sequence (order matters)
Transformer Blocks (×N)	The core processing layers — each block refines the model's understanding of every token
Self-Attention (inside each block)	Allow every token to look at every other token and gather relevant context
Feed-Forward Network (inside each block)	Apply learned transformations to each token's updated representation
Layer Normalisation	Stabilise the signal passing through each layer during training
Output Layer (LM Head)	Convert the final token representations into probability scores over the vocabulary

3.2 Encoder vs. Decoder Transformers

Type	Architecture	Best Known Models & Uses
Encoder-only	Reads the full input bidirectionally — sees all tokens at once	BERT, RoBERTa — Classification, search, embeddings
Decoder-only	Reads left to right — each token only sees what came before it	GPT-4, Claude, Llama — Text generation, chatbots, code
Encoder-Decoder	Encoder reads input; decoder generates output step by step	T5, BART — Translation, summarisation, Q&A

NOTE

Most modern LLMs (GPT-4, Claude, Gemini, Llama) are decoder-only Transformers. When people say 'LLM', they almost always mean a decoder-only model.

3.3 How Deep is a Transformer?

A single Transformer block is stacked many times to create depth. Each layer allows the model to build progressively more abstract representations.

Model	Transformer Layers (Depth)
GPT-2 (small, 117M params)	12 layers
BERT-Large	24 layers
GPT-3 (175B)	96 layers
GPT-4 (estimated)	~120 layers (with mixture of experts)
Llama 3 70B	80 layers
What each layer adds	Earlier layers: syntax and local patterns. Later layers: semantics, reasoning, world knowledge

3.4 The Feed-Forward Network (FFN)

Inside each Transformer block, after self-attention, is a Feed-Forward Network — two linear transformations with a non-linear activation in between.

KEY CONCEPT

The FFN applies the same learned transformation to each token position independently. It is thought to act as a 'memory' — storing factual associations learned during training. Research suggests specific facts ('Paris is the capital of France') are stored in specific FFN neurons.

The FFN is typically 4× larger than the attention layer — it accounts for ~⅓ of a model's parameters.

KEY TAKEAWAY

The Transformer: embed tokens → add position → run through N blocks of (attention + FFN + normalisation) → project to vocabulary probabilities.

Module 4: The Attention Mechanism

The attention mechanism is the heart of the Transformer and the single most important concept in modern AI. It is what allows LLMs to understand context across long passages of text.

4.1 The Problem Attention Solves

To understand a word, you need context. The word 'bank' means something different in 'river bank' and 'bank account'. Older models (RNNs) struggled to use context from far earlier in a sentence. Attention solves this completely.

ANALOGY

Imagine you are at a large conference. When someone mentions 'the merger', you automatically scan the room and think about the most relevant people and prior conversations. You do not pay equal attention to everyone — you weight your attention toward what is most relevant to understanding 'the merger' in context. That is exactly what self-attention does: for every word, it decides how much attention to pay to every other word in the sequence.

4.2 Self-Attention: Step by Step

Step	What Happens
1. Create Q, K, V	For each token embedding, create three vectors: Query (Q), Key (K), and Value (V) using learned weight matrices
2. Compute Attention Scores	For each token's Query, compute a dot product with every other token's Key. High score = high relevance.
3. Scale and Softmax	Divide scores by $\sqrt{\text{dimension}}$ to prevent exploding values. Apply softmax to get attention weights that sum to 1.
4. Weighted Sum of Values	Multiply each token's Value by its attention weight. Sum these up. This becomes the new, context-enriched representation.
5. Output	Each token now has a new embedding that has 'seen' and incorporated relevant information from all other tokens.

```
# Self-Attention Intuition - Sentence: 'The animal didn't cross the street because it was tired'

Token:      'it'
Query:      What is this token asking about?
            → 'What does it refer to?'

Keys:       What does each other token offer?
            'animal' → high relevance score
            'street' → lower relevance score
```


'tired' → high relevance score

Result: 'it' now has a representation enriched with information from 'animal' and 'tired'.
The model now understands 'it' refers to the animal.

4.3 Multi-Head Attention

Instead of running one attention pass, Transformers run multiple attention operations in parallel — each with its own Q, K, V weight matrices. These are called 'heads'.

KEY CONCEPT

Each attention head learns to focus on different types of relationships simultaneously:

- Head 1 might track grammatical subject-verb relationships
- Head 2 might track pronoun-antecedent references
- Head 3 might track semantic similarity between concepts
- Head 4 might track positional proximity

The outputs of all heads are concatenated and projected to produce the final context-enriched representation.

Model	Number of Attention Heads
BERT-Base	12 heads
GPT-3	96 heads
Llama 3 8B	32 heads
Llama 3 70B	64 heads
Why more heads?	More heads = more relationship types the model can track simultaneously

4.4 Causal (Masked) Attention

In decoder-only LLMs (GPT, Claude, Llama), a critical modification is applied: each token can only attend to previous tokens, never future ones. This is called causal or masked attention.

KEY CONCEPT

Why causal masking? Because during training, the model must predict the NEXT token. If it could see future tokens, the task would be trivially easy — and the model would learn nothing. During generation, masking is not needed — there are no future tokens yet. But the model was trained with masking, so it generates autoregressively: one token at a time.

4.5 The Context Window

The context window is the maximum number of tokens the model can 'see' at once during a single forward pass. It is the model's working memory.

Model	Context Window
GPT-3 (original)	2,048 tokens (~1,500 words)
GPT-4 Turbo	128,000 tokens (~96,000 words)
Claude 3.5 Sonnet	200,000 tokens (~150,000 words)
Gemini 1.5 Pro	1,000,000 tokens (~750,000 words — entire books)
Llama 3.1 405B	128,000 tokens
Practical impact	Longer context = more expensive, slower, but better for long documents and multi-step reasoning

4.6 Positional Encoding

Attention has no inherent sense of order — it treats all positions equally. Positional encoding injects sequence order into the model.

Method	How It Works & Who Uses It
Sinusoidal (original)	Add a fixed sine/cosine wave pattern to each token embedding. Simple, works for seen sequence lengths. Used in original Transformer.
Learned Absolute (APE)	Learn a separate embedding for each position during training. Used in BERT, GPT-2.
Rotary (RoPE)	Rotate token embeddings by an angle proportional to position. Excellent generalisation to longer sequences. Used in Llama, Mistral, GPT-NeoX.
ALiBi	Add a linear bias to attention scores based on distance. Simple, generalises to longer sequences without retraining. Used in MPT, Falcon.

KEY TAKEAWAY

Attention = each token gathers relevant context from all other tokens, weighted by relevance. Multi-head attention does this for many relationship types simultaneously. Causal masking prevents future-peeking during training.

Module 5: Pre-Training — How LLMs Learn

Pre-training is where the model learns from vast amounts of text. It is the most computationally expensive phase — and the most impactful.

5.1 The Pre-Training Objective

KEY CONCEPT

The core training task is deceptively simple: given all the preceding tokens, predict the next token.

This is called Causal Language Modelling (CLM) for decoder models, or Masked Language Modelling (MLM) for encoder models.

The model sees text, makes predictions, compares to the actual next token, measures the error (cross-entropy loss), and adjusts its weights via backpropagation.

This is repeated billions of times across the entire training corpus.

5.2 The Training Data

Modern LLMs are trained on text corpora of extraordinary scale and diversity.

Data Source	Characteristics & Examples
Common Crawl	A web crawl of billions of pages. The largest single source. Noisy — requires heavy filtering.
Books / Literature	BookCorpus, Project Gutenberg, internet book collections. High-quality long-form reasoning.
Wikipedia	Encyclopaedic factual knowledge across many languages. High quality, well-structured.
Code (GitHub, Stack Overflow)	Source code in dozens of languages. Teaches syntax, logic, and problem-solving structure.
Scientific papers (arXiv, PubMed)	Specialised domain knowledge. Improves reasoning and factual accuracy in STEM.
Reddit, forums, Q&A sites	Conversational patterns, reasoning, opinions. Teaches dialogue and informal register.
News articles	Current events (up to cutoff), journalistic style, factual reporting.
Synthetic data	LLM-generated data used to bootstrap capability in specialised domains.

5.3 Training Scale

Model	Training Tokens
GPT-3 (2020)	300 billion tokens

LLaMA 1 (2023)	1.4 trillion tokens
Llama 3 (2024)	15 trillion tokens
GPT-4 (2023)	Undisclosed (estimated ~13T tokens)
Gemini Ultra (2023)	Undisclosed
Cost to train GPT-4	—

5.4 The Training Loop — Step by Step

Step	Detail
1. Sample a batch	Select a batch of token sequences from the training data (typically 2–4 million tokens per batch)
2. Forward pass	Run each sequence through the full Transformer — all N layers — to produce next-token predictions
3. Compute loss	Calculate cross-entropy loss: how different are the predicted probability distributions from the actual next tokens?
4. Backward pass (backprop)	Calculate the gradient of the loss with respect to every parameter in the model
5. Update weights	Apply an optimiser (AdamW) to nudge each parameter slightly in the direction that reduces loss
6. Repeat	Repeat billions of times across the entire training corpus, often for multiple epochs
7. Learning rate schedule	The learning rate (step size) is carefully scheduled: warm-up, then cosine decay to zero

5.5 What the Model Learns During Pre-Training

Capability	How It Emerges from Next-Token Prediction
Grammar and syntax	To predict the next word, you must understand sentence structure
Factual knowledge	Facts appear repeatedly — the model learns to associate them
Reasoning patterns	Logical and causal chains appear throughout text — the model learns their structure
Code understanding	Code is just structured text — the same prediction objective applies
Multi-lingual ability	Text from many languages trains understanding of all of them simultaneously
Analogies and metaphors	These are everywhere in text — the model learns conceptual relationships

In-context learning	Emerges at scale — the model learns to use examples provided in the prompt
---------------------	--

5.6 The Scaling Laws

One of the most important empirical discoveries in AI: model performance improves predictably as you scale up model size, data, and compute — following a power law.

KEY CONCEPT

Chinchilla Scaling Laws (Hoffmann et al., 2022): For optimal training, scale model size and training tokens together. For every doubling of parameters, roughly double the training data. Practical implication: Many early models (including GPT-3) were under-trained on data relative to their parameter count. Llama 2/3 trains smaller models on far more data — achieving better performance per parameter.

Returns are not diminishing — they are predictable. This is why AI labs invest billions in training runs with confidence.

KEY TAKEAWAY

Pre-training = showing the model billions of next-token prediction problems across diverse text. The model learns language, knowledge, and reasoning as a side effect of solving this prediction task at scale.

Module 6: Fine-Tuning, RLHF, and Alignment

A pre-trained model is knowledgeable but raw — it completes text patterns but is not yet a helpful assistant. The post-training phase transforms it into the models we use.

6.1 The Three Phases of Post-Training

Phase	What It Does	Output
Supervised Fine-Tuning (SFT)	Train on high-quality (instruction, response) pairs written or curated by humans	Model that follows instructions and produces coherent conversational answers
Reward Modelling (RM)	Train a separate model to predict human preferences between two responses	A 'judge' model that scores responses for helpfulness, safety, honesty
RLHF / PPO or DPO	Use the reward model to further fine-tune the LLM via reinforcement learning	A helpful, safe, honest assistant aligned with human values

6.2 Supervised Fine-Tuning (SFT)

SFT teaches the model to follow instructions by training on thousands of high-quality examples of the format: instruction → ideal response.

```
# SFT Training Example

Input:  'Explain Newton's second law of motion in simple terms
        for a high school student.'

Target: 'Newton's second law states that force equals mass
        times acceleration (F = ma). In plain English:
        heavier objects need more force to accelerate, and
        the same force accelerates lighter objects faster.
        Example: pushing a shopping cart — a full cart
        needs more push than an empty one.'
```

```
# The model learns: when given an instruction, produce a
# helpful, complete, appropriate response.
```

6.3 Reinforcement Learning from Human Feedback (RLHF)

RLHF is the technique that turned raw language models into aligned assistants. It uses human feedback to train the model to be more helpful, harmless, and honest.

RLHF Step	Detail
-----------	--------

Step 1: Collect comparisons	Human raters see two model responses to the same prompt and choose which is better
Step 2: Train reward model	A separate neural network (the reward model) is trained to predict which response humans prefer
Step 3: RL fine-tuning	The LLM is further trained using PPO (Proximal Policy Optimisation) — maximising the reward model's score
Step 4: KL constraint	A penalty prevents the model from drifting too far from the original SFT model (prevents reward hacking)
Result	A model that produces responses humans find more helpful, safe, and honest — without being directly told what 'good' means

6.4 Direct Preference Optimisation (DPO)

DPO is a newer, simpler alternative to RLHF that achieves similar alignment results without a separate reward model or RL training loop.

KEY CONCEPT

DPO directly optimises the policy (the LLM) using preference pairs — without needing a separate reward model or RL algorithm.

For each (prompt, chosen_response, rejected_response) triple, DPO increases the probability of the chosen response and decreases the probability of the rejected one.

DPO is more stable, simpler to implement, and computationally cheaper than RLHF. It is now the dominant fine-tuning approach for open-source models.

Used by: Llama 3, Mistral Instruct, Zephyr, Phi-3, and most modern open-source instruction-following models.

6.5 Constitutional AI (CAI) — Anthropic's Approach

Anthropic developed Constitutional AI as an alternative to pure RLHF. It uses a set of principles (a 'constitution') to guide the model's own self-critique during training.

CAI Step	What Happens
1. Generate response	Model generates an initial response to a potentially harmful prompt
2. Critique	Model critiques its own response against a list of principles (harmlessness, honesty, helpfulness)
3. Revise	Model revises its response to better satisfy the principles
4. RLAIIF	Use AI feedback (instead of only human feedback) to train a reward model on these revised responses
5. Fine-tune	Fine-tune the main model using RL with this AI-feedback reward model

6.6 Alignment — The Bigger Picture

Goal	Description	Technique
Helpfulness	Model provides genuinely useful, complete responses	SFT on high-quality examples; RLHF reward signal
Harmlessness	Model refuses to help with dangerous, illegal, or harmful requests	Safety-focused SFT; red-teaming; constitutional principles
Honesty	Model acknowledges uncertainty; doesn't deceive; won't claim to be human	Calibration training; honesty-focused SFT examples
Instruction Following	Model does what the user asks, within safety constraints	High-quality instruction SFT data; diverse task coverage
Value Alignment	Model's priorities match human values in novel situations	RLHF; CAI; ongoing red-teaming and model evaluation

KEY TAKEAWAY

Pre-training gives the model knowledge. SFT gives it instructions-following behaviour. RLHF / DPO / CAI align it with human values. These three phases together produce the assistants we use.

Module 7: Inference — How LLMs Generate Text

Inference is the process of using a trained model to generate a response. Understanding it explains why LLMs have the latency they do and why outputs are probabilistic.

7.1 Autoregressive Generation

KEY CONCEPT

LLMs generate text one token at a time. Each token is generated by running the full model forward and sampling from the output probability distribution.

Each newly generated token is appended to the context, and the model runs again to produce the next token.

This is called autoregressive generation — each output depends on all previous outputs.

Practical implication: Generating 100 tokens requires 100 separate forward passes through the full model.

```
# Autoregressive Generation — Simplified

Input context: 'The capital of France is'

Step 1: Model outputs probability distribution over vocabulary
        'Paris' → 94.2%   ' Paris' → 3.1%   'Lyon' → 0.8%   ...
        → Sample: 'Paris'

Step 2: Context: 'The capital of France is Paris'
        'and' → 12%     ', ' → 34%     ' .' → 28%     ...</p>
        → Sample: ' .'

Step 3: Context: 'The capital of France is Paris .'
        '<end>' → 87%    'It' → 6%     'The' → 4% ...
        → Sample: '<end>' → generation stops
```

7.2 Sampling Strategies

How a token is selected from the probability distribution dramatically affects output quality and creativity.

Strategy	How It Works & When to Use
Greedy Decoding	Always pick the highest-probability token. Fast, deterministic, but often repetitive and bland.
Temperature Sampling	Divide all logits by a temperature T before softmax. T < 1 = more focused / deterministic. T > 1 = more random / creative. T = 1 = default.
Top-K Sampling	Only sample from the K highest-probability tokens. Prevents very unlikely tokens. K=50 is common.

Top-P (Nucleus) Sampling	Sample from the smallest set of tokens whose cumulative probability exceeds P (e.g. P=0.9). Adapts to the distribution dynamically.
Beam Search	Maintain N candidate sequences in parallel. Pick the overall highest-probability sequence. Used in translation; rarely in chat.
Typical sampling	Samples tokens near the median information content — avoids both the most and least likely tokens. Used in some creative applications.

7.3 The KV Cache

Running the full model forward for every new token is slow. The KV Cache (Key-Value Cache) dramatically speeds up generation by storing and reusing intermediate computations.

KEY CONCEPT

During self-attention, every token computes Keys and Values for all preceding tokens.

Without caching: recompute ALL K/V pairs every step. For a 100-token context, each new token recomputes 100 K/V pairs.

With KV Cache: store the K/V pairs from previous steps. Each new step only computes K/V for the ONE new token.

Result: ~10–100× faster generation. Almost all production inference systems use KV caching.

Cost: KV cache consumes significant GPU memory — this limits the practical context window at scale.

7.4 Inference Optimisation Techniques

Technique	What It Does
Quantisation	Reduce parameter precision from float32 → float16 → int8 → int4. 4× memory reduction with small quality loss. Enables running large models on consumer hardware.
Speculative Decoding	A small 'draft' model proposes K tokens; the large model verifies them in one pass. 2–3× speedup with identical output quality.
Batching	Process multiple requests simultaneously on the same GPU. Maximises hardware utilisation for API providers.
Flash Attention	A memory-efficient algorithm for computing attention. Same mathematical result, ~2–4× faster, far less memory.
Continuous Batching	Dynamically add new requests to a batch as earlier ones complete. Near-100% GPU utilisation for high-throughput serving.
Tensor Parallelism	Split the model across multiple GPUs. Required for models too large for a single GPU's memory.

7.5 Prompt Processing vs. Token Generation

Phase	What Happens & Speed
Prefill (Prompt Processing)	The full prompt is processed in parallel in one forward pass. Very fast — all tokens processed simultaneously.
Decode (Generation)	Tokens generated one at a time, autoregressively. Slower — one forward pass per output token.
Time to First Token (TTFT)	Time to process the prompt and generate the first output token. Affected by prompt length.
Tokens per Second (TPS)	Speed of the generation phase. Typical commercial APIs: 50–150 tokens/sec.
Why prompt length matters	A 10,000-token prompt takes ~10× longer to process than a 1,000-token prompt in the prefill phase.

KEY TAKEAWAY

Inference = autoregressive token-by-token generation using sampling from probability distributions. The KV cache is the key optimisation that makes this practical at scale.

Module 8: Capabilities, Limitations, and Hallucinations

8.1 What LLMs Are Genuinely Good At

Capability	Why LLMs Excel Here
Language understanding	Trained on more text than any human reads in thousands of lifetimes
Text generation and writing	Vast exposure to every writing style, genre, and format
Summarisation	Combines language understanding with the ability to identify salient information
Translation	Trained on parallel multilingual text; strong for common language pairs
Code generation	Code is well-structured text with clear patterns; pre-training on GitHub is highly effective
Explaining concepts	Seen thousands of explanations at every difficulty level
Brainstorming and ideation	Pattern-matching across diverse domains produces novel combinations
Instruction following	Post-training specifically optimises this capability
Few-shot learning	Emergent ability: the model adapts to new tasks from just a few examples in the prompt

8.2 What LLMs Struggle With

Limitation	Root Cause
Arithmetic and precise calculation	Tokenisation treats numbers as text; no internal calculator; pattern-matching fails for large numbers
Strict logical reasoning	Each token prediction is local; long chains of strict logic can accumulate errors step by step
Spatial and physical reasoning	Very little spatial information in text; the model cannot 'simulate' physical reality
Real-time / post-cutoff information	Knowledge is frozen at training cutoff; no live data access (unless a tool is provided)
Long-range planning	Autoregressive generation is forward-only; no global planning before writing
Counting and exact enumeration	Tokens are not discrete countable objects internally; exact counting is unreliable

Consistently following very long instructions	Attention to constraint details degrades in very long system prompts
Self-knowledge	The model has limited reliable insight into its own parameters or training

8.3 Hallucinations — The Core Problem

KEY CONCEPT

Hallucination: When an LLM generates plausible-sounding but factually incorrect or fabricated information with apparent confidence.

Root cause: The model is optimised to generate probable next tokens, not to verify factual accuracy. A plausible-sounding fabrication is 'rewarded' the same as truth during training.

Hallucination Type	Description	Example
Factual error	States a false fact confidently	Claims a real person said something they never said
Entity fabrication	Invents people, places, publications that do not exist	Cites a research paper that was never written
Numeric confabulation	Gives wrong statistics or calculations	States the wrong founding year of a real company
Source invention	Makes up URLs, book titles, or quotes	Provides a plausible-looking but fake citation
Confident ignorance	Gives a wrong answer when it should say 'I don't know'	Answers a niche question incorrectly rather than admitting uncertainty

Mitigation Strategy	How It Reduces Hallucination
Retrieval-Augmented Generation (RAG)	Ground the model in retrieved source documents; instruct it to cite only what is in the context
Chain-of-Thought Prompting	Step-by-step reasoning exposes errors before the final answer; more visible and correctable
Temperature = 0	Deterministic, highest-probability sampling reduces creative-but-wrong confabulation
Calibration training	Train models to express uncertainty ('I am not sure', 'I may be wrong here')
External verification	Always fact-check high-stakes outputs with authoritative external sources
Constitutional AI / RLHF	Trains the model to prefer honest, hedged responses when uncertain

8.4 The Context Window and 'Lost in the Middle'

Even with large context windows, LLMs do not attend equally to all parts of their context. Research shows a systematic bias:

KEY CONCEPT

Lost in the Middle (Liu et al., 2023): LLMs perform best on information at the START and END of long contexts. Information in the middle of a very long context is often 'forgotten' or attended to less reliably.

Practical guidance: Put the most critical instructions at the beginning of the system prompt and repeat key constraints near the end. Do not bury critical information in the middle of a 100,000-token context.

KEY TAKEAWAY

LLMs are extraordinary language and knowledge systems — but they are not infallible. Understanding their failure modes (hallucination, context limitations, arithmetic) is as important as understanding their strengths.

Module 9: Leading LLMs Compared

The LLM landscape changes rapidly. This module provides a snapshot of the most important models, their architectures, and their strengths.

9.1 Commercial Frontier Models

Model	GPT-4o (OpenAI)	Claude 3.5 Sonnet (Anthropic)	Gemini 1.5 Pro (Google)
Architecture	Decoder-only Transformer (MoE suspected)	Decoder-only Transformer	Decoder-only Transformer (MoE)
Context Window	128K tokens	200K tokens	1M tokens
Modalities	Text, image, audio, video	Text, image	Text, image, audio, video
Strengths	Code, tool use, broad capability, real-time voice	Long context, nuanced writing, instruction following, safety	Very long context, multimodal, Google ecosystem
Training Approach	Pre-train + RLHF + additional alignment	Pre-train + RLHF + Constitutional AI	Pre-train + RLHF + RLAIF
Knowledge Cutoff	April 2024	April 2024	November 2023
Best For	General use, agents, coding, APIs	Document analysis, writing, long-context tasks	Long video/audio analysis, very large documents

9.2 Open-Source Frontier Models

Model (Organisation)	Key Characteristics	Best Use Case
Llama 3.1 405B (Meta)	405B params; trained on 15T tokens; Apache 2.0 licence; strong general capability	Commercial use requiring data privacy; self-hosted LLM applications
Llama 3.1 8B / 70B (Meta)	Smaller variants; 70B is competitive with many closed models; fast to run	Edge deployment; cost-sensitive applications; fine-tuning base models
Mistral Large 2 (Mistral AI)	Strong code and reasoning; efficient architecture; European model	Code generation; instruction following; EU data residency requirements
Mixtral 8×22B (Mistral AI)	Mixture-of-experts; activates only 2 of 8 expert networks per token; very efficient	High-throughput, cost-effective deployment with strong performance
Qwen 2.5 72B (Alibaba)	Excellent multilingual performance, especially Chinese; strong coding	Multilingual applications; Asian market products; code generation

Phi-3.5 (Microsoft)	Very small (3.8B params) but surprisingly capable; runs on phones	On-device AI; mobile applications; resource-constrained environments
DeepSeek-V3 (DeepSeek)	Highly efficient training (reported \$6M cost); strong coding and reasoning	Cost-effective reasoning tasks; alternatives to larger closed models

9.3 Specialised and Reasoning Models

Model	Specialisation & Notes
o1 / o3 (OpenAI)	Reasoning-optimised models. Use extended 'thinking time' before answering. Excel at maths, science, complex logic at the cost of higher latency and cost.
o1-mini (OpenAI)	Efficient reasoning model. Significantly cheaper than o1 with strong performance on coding and STEM tasks.
Claude 3 Opus (Anthropic)	Anthropic's most capable model for complex analysis and nuanced tasks requiring deep reasoning.
Gemini 1.5 Flash (Google)	Optimised for speed and cost. Retains 1M context window. Excellent for high-volume, latency-sensitive applications.
Code Llama (Meta)	Fine-tuned from Llama specifically for code generation, debugging, and code explanation.
Meditron (EPFL)	Fine-tuned on medical literature. Specialised for clinical reasoning and medical Q&A.

9.4 Key Architectural Innovations

Innovation	Description & Impact
Mixture of Experts (MoE)	Instead of activating all parameters for every token, route each token to a subset of 'expert' networks. GPT-4, Mixtral, Gemini use MoE. Achieves large model capability at lower inference cost.
Grouped Query Attention (GQA)	Share Key/Value heads across multiple Query heads. Dramatically reduces KV cache memory. Used in Llama 2/3, Mistral.
Sliding Window Attention	Some attention heads only attend to nearby tokens; others attend globally. Reduces memory for long contexts. Used in Mistral.
Flash Attention 2/3	Hardware-aware attention implementation. Same result, 2–4× faster, 10× less memory. Now standard across all major training frameworks.
Extended Thinking / Chain-of-Thought Tokens	Models generate internal reasoning tokens ('thinking') before the final answer. o1, Claude 3.7, DeepSeek-R1 use this. Dramatically improves complex reasoning.

Speculative Decoding	Small draft model proposes tokens; large model verifies. 2–3× generation speedup with identical outputs.
----------------------	--

KEY TAKEAWAY

The model landscape is competitive and fast-moving. The key differentiators are: context window length, multimodal capability, reasoning depth, cost per token, and data privacy/hosting options.

Module 10: The Future of LLMs

10.1 Scaling Laws and Their Limits

One of the most debated questions in AI: will scaling continue to produce predictable capability improvements, or are we approaching a wall?

Perspective	Evidence & Argument
Scaling will continue to deliver	GPT-4 to o1/o3 shows continued gains; Chinchilla laws suggest we are still data-limited; new architectures (MoE) are more efficient; inference-time scaling (thinking tokens) is an entirely new axis.
Diminishing returns emerging	Some benchmarks show saturation; web-scale data is being exhausted; gains are more expensive per unit; some capabilities may require architectural breakthroughs, not just scale.
Inference-time scaling is the new frontier	o1/o3 and DeepSeek-R1 show that spending more compute at inference (thinking longer) can substitute for more training. This is a new scaling axis.

10.2 Key Research Directions

Direction	What Is Being Researched
Reasoning and planning	Teaching models to reason more reliably: extended thinking, process reward models, planning algorithms
Long-context and memory	Extending effective context windows; persistent memory across sessions; external memory architectures
Multimodality	Natively multimodal training from scratch (not bolted-on); video understanding; robotic embodiment
Efficiency	Smaller models that match larger ones via better data and training; quantisation; distillation
Agents	Reliable tool use; long-horizon planning; multi-agent collaboration; computer-use agents
Interpretability	Understanding what is happening inside models: circuits, features, concepts (Anthropic's mechanistic interpretability work)
Alignment at scale	Ensuring increasingly capable models remain safe and aligned as capability increases
Synthetic data	LLMs generating their own training data; self-improvement without new human-labelled data

10.3 The Road to AGI

Stage	Description	Current Status
Narrow AI	Superhuman at specific tasks (chess, protein folding, image classification)	Achieved — clearly surpassed
Emerging AGI	Broadly capable across many domains; handles novel tasks with reasoning	Current frontier — GPT-4, Claude 3.5 level
AGI	Can perform any intellectual task a human can, at human level or above	Not yet achieved — timeline disputed (2027–2035+ range)
Superhuman Reasoning	Consistently outperforms the best humans on scientific reasoning, maths, coding	Approaching in narrow domains (o3 on ARC-AGI)
Full Autonomy	Can set and pursue complex long-term goals in the real world reliably	Not yet achieved — significant reliability challenges remain

10.4 What This Means for Practitioners

- **Build on APIs, not from scratch:** Training frontier models costs \$50–100M+. The skill is in applying them effectively.
- **Prompt engineering remains the highest-leverage skill:** Regardless of model improvements, knowing how to communicate with LLMs effectively is critical.
- **Agent development is the next major wave:** Models with tool use and planning are transforming workflows. Learn to build agents.
- **Fine-tuning is democratised:** LoRA and QLoRA make it possible to specialise powerful open-source models on a single consumer GPU.
- **Evaluation is undervalued:** As capabilities grow, knowing how to measure model performance on your specific task is a major competitive advantage.
- **Follow the research:** The field moves fast. arXiv, Anthropic's blog, OpenAI papers, and AI Safety research are primary sources.

KEY TAKEAWAY

LLMs are not magic — they are a mathematical function (a very large one) optimised to predict text.
The magic is emergent: simple objectives at massive scale produce capabilities that feel like intelligence.
The most important insight: understanding the mechanics helps you use LLMs better, build better products, and anticipate what comes next.

Quick Revision Reference

Glossary of Key Terms

Term	Definition
LLM	Large Language Model — a Transformer neural network trained on massive text corpora to predict the next token
Token	The basic unit of text processed by an LLM. Roughly 0.75 words; a vocabulary of 30K–100K tokens.
Embedding	A dense vector of floating-point numbers representing the meaning of a token or sequence
Transformer	The neural network architecture (2017) underlying all modern LLMs; uses attention as its core mechanism
Self-Attention	Mechanism allowing each token to gather context from all other tokens, weighted by relevance
Multi-Head Attention	Running multiple attention operations in parallel, each learning different relationship types
Causal Masking	Restricting each token to only attend to previous tokens; essential for autoregressive generation
Positional Encoding	Information added to embeddings to represent token order in the sequence
Feed-Forward Network (FFN)	Per-token transformation layer inside each Transformer block; stores factual knowledge
Context Window	Maximum tokens the model can process at once — the model's working memory
BPE (Byte-Pair Encoding)	Tokenisation algorithm that builds a vocabulary of common sub-word units from a corpus
Pre-training	Initial training on massive text corpora using next-token prediction objective
Fine-tuning	Further training on smaller, task-specific datasets after pre-training
SFT (Supervised Fine-Tuning)	Training on (instruction, ideal response) pairs to teach the model to follow instructions
RLHF	Reinforcement Learning from Human Feedback — aligns models with human preferences using a reward model
DPO	Direct Preference Optimisation — simpler alternative to RLHF; directly optimises on preference pairs
Constitutional AI (CAI)	Anthropic's alignment approach using a set of principles for model self-critique during training
Autoregressive Generation	Generating text one token at a time, each conditioned on all previous tokens

Temperature	Sampling parameter — lower = more deterministic, higher = more random/creative
Top-P (Nucleus) Sampling	Sample from smallest set of tokens whose cumulative probability exceeds threshold P
KV Cache	Stores Key/Value attention computations from earlier tokens to avoid recomputation; enables fast generation
Hallucination	Model generating plausible-sounding but factually incorrect information confidently
Scaling Laws	Empirical power-law relationship between model size, data, compute, and performance
MoE (Mixture of Experts)	Architecture routing each token to a subset of specialised networks; efficient use of parameters
Quantisation	Reducing numerical precision of model weights to decrease memory and increase inference speed
RAG (Retrieval-Augmented Generation)	Grounding LLM outputs in retrieved external documents to reduce hallucination
In-Context Learning	Emergent ability to perform new tasks from examples provided in the prompt without retraining
Inference-Time Scaling	Spending more compute at generation time (extended thinking) to improve output quality
Knowledge Cutoff	The date after which the model has no training data — has no knowledge of later events

The LLM Pipeline — End to End

Stage	Summary
1. Data Collection	Collect trillions of tokens from web, books, code, scientific papers
2. Data Cleaning	Filter, deduplicate, quality-score, and mix data sources carefully
3. Tokenisation	Build a BPE vocabulary; convert all text to integer token sequences
4. Pre-Training	Train Transformer on next-token prediction across the full corpus for months on thousands of GPUs
5. Supervised Fine-Tuning	Fine-tune on high-quality instruction-response pairs
6. Reward Modelling	Train reward model on human preference comparisons
7. RLHF / DPO	Fine-tune using reward signal to maximise helpfulness, safety, honesty
8. Evaluation	Benchmark on standard evals; red-team for safety; human evaluation
9. Deployment	Serve via API with KV caching, batching, quantisation for efficiency
10. Monitoring	Track outputs, collect feedback, iterate on alignment and capability

Further Learning Resources

Resource	Description
'Attention Is All You Need' (Vaswani et al. 2017)	The original Transformer paper — mandatory reading for understanding the architecture
Andrej Karpathy: 'Let's build GPT from scratch'	YouTube tutorial — best practical introduction to LLM internals; builds a GPT from scratch in Python
Jay Alammar: 'The Illustrated Transformer'	jalammar.github.io — beautiful visual explanations of attention and the Transformer
Sebastian Raschka: 'Build a Large Language Model from Scratch'	Full book available free online — hands-on implementation guide
DeepLearning.AI: Transformers short course	Free short course on Transformer architecture with practical code
Andrej Karpathy: 'Intro to Large Language Models'	YouTube talk — excellent conceptual overview of how LLMs work (1 hour)
Chinchilla Scaling Laws paper (Hoffmann et al. 2022)	Key paper on optimal compute allocation for LLM training
Anthropic Interpretability Research	anthropic.com/research — cutting-edge work on understanding what is inside LLMs
RLHF paper (InstructGPT — Ouyang et al. 2022)	The paper that introduced RLHF for LLM alignment
Hugging Face Transformers documentation	huggingface.co/docs — practical guide to using and fine-tuning open-source LLMs

AI & Data Science Study Series | How Large Language Models Work
Study. Build. Grow.